

메시지 알림 라우터 — 에이전트 워크플로우 & 엔지니어링 보고서

HackerRank Orchestrate, 2026년 8월 — AI Judge 인터뷰 대비 자료. 이번 세션의 실제 구현·테스트·의사결정 기록을 바탕으로 작성.

1. 요약

과제: WhatsApp과 유사한 플랫폼을 위한 개인화된 알림 라우터를 구축하는 것. 수신되는 모든 메시지(텍스트, 이미지 포스터/스크린샷, 음성 메모)에 대해 notify(즉시 알림), digest(나중에 표시), mute(억제) 중 하나를 판단하고, message_type, 근거가 담긴 reason, 보정된 confidence, 사용자 본인 이력에서 가져온 evidence_message_ids까지 산출해야 한다 — 메시지 내용만으로는 아니라 수신자별로 개인화된 판단이 핵심이다.

접근 방식: 로컬 전처리 계층(SQLite 기반 컨텍스트 구성, 캐시된 이미지/음성 분석)이 JSON 스키마로 제약된 Claude Sonnet 5 분류 단계에 데이터를 공급한다. 프롬프트와 스키마는 8회의 기록된 반복을 거쳐 다듬어졌으며, 매번 추측이 아니라 실제 오답 사례를 직접 읽고 근거를 도출했고, 세 가지 독립적인 신호로 검증했다: 공식 정답 30건, 보이는 예시에 과적합했는지 확인하기 위해 전용으로 만든 진짜 held-out 합성 테스트 19건(텍스트 + 생성 이미지 + 생성 음성), 그리고 이미 어려운 것으로 확인된 케이스만 저렴하게 재검증하는 타겟 체크.

action 정확도	93.33% (30건 정답 세트 기준)
message_type 정확도	93.33% (30건 정답 세트 기준)
evidence F1	~48–52% (여러 평가 기준 공통)
output.csv	110/110행, 스키마 검증 통과, fallback 행 없음
모델	claude-sonnet-5 (비전+분류), 로컬 faster-whisper (음성 인식)

인터뷰에서 가장 의미 있게 볼 만한 사실은 오히려 “실패 사례”일 수 있다: 그럴듯해 보이던 프롬프트 아이디어(key_phrase grounding 필드)를 실제로 구현·테스트했더니 정확도가 퇴보했고(93.33% → 86.67%), 이를 발견해 되돌렸다 — 이런 사례를 굳이 이 보고서에 남긴 이유는, 나쁜 아이디어를 잡아내고 되돌리는 것도 채택된 개선 못지않게 프로세스 품질을 보여주는 신호이기 때문이다.

2. 문제 요약

- 입력: dataset/messages.csv — 수신 메시지 1건당 1행 (발신자, 대화 유형, 타임스탬프, 텍스트 또는 미디어 참조).
- 출력: message_id, action, message_type, reason, confidence, evidence_message_ids — 정확한 컬럼, 정확한 순서, 입력 메시지당 정확히 1행.
- 개인화가 핵심 요구사항이다: 동일한 메시지 내용이라도 수신자의 관계·이력·선호도에 따라 한 사람에겐 notify, 다른 사람에겐 mute가 될 수 있다.
- 멀티모달: 텍스트, 이미지 포스터/스크린샷(OCR/비전 필요), 음성 메모(ASR 필요)를 모두 실제로 분석해야 하며 임의의 기본값으로 처리하면 안 된다.
- 제약사항: 하드코딩된 라벨이나 파일별 특화 답변 금지; 가능한 한 결정론적으로 동작; 비밀값은 환경변수로만; 평가 워크플로우 필수 포함.
- 제공된 컨텍스트 데이터: 사용자별 알림 행동, 그룹 메타데이터와 멤버십, 비즈니스 계정 신뢰 신호 및 사용자별 관계 이력, 과거 메시지와 그에 대한 사용자 반응(열람/답장/무시/음소거/신고), 일일 알림량.

3. 시스템 아키텍처

파이프라인은 판단 로직과 실행 로직을 5단계로 분리하며, 하나의 거대한 프롬프트가 모든 걸 처리하는 대신 각 단계를 독립적으로 테스트 가능한 별도 모듈로 구성했다:

단계	모듈	역할
1. 데이터 적재	db.py	dataset/*.csv 전체를 로컬 SQLite DB로 적재, 실행마다 새로 빌드. dataset/은 output.csv를 제외하면 엄격히 읽기 전용.
2. 미디어 정규화	image_pipeline.py audio_pipeline.py	이미지: 확장자를 신뢰하지 않고 실제 MIME 타입 감지 → Pillow로 디코드 → 리사이즈 → JPEG 재인코딩 → Claude에 보내 설명+OCR+문서유형 산출. 음성: 교체 가능한 provider 인터페이스 뒤에서 로컬 faster-whisper로 전사 (Messages API에는 직접적인 오디오 입력 타입이 없음).
3. 컨텍스트 구성	router.py pull_context()	메시지마다: 수신 사용자 프로필, 해당 그룹멤버십 또는 비즈니스관계 행, 캐시된 미디어 분석, 오늘의 알림량, 해당 사용자 본인 메시지 이력 중 관련성 순으로 추린 일부(반응 포함) — 전체 CSV를 절대 통째로 보내지 않음.
4. 분류	router.py classify_message()	JSON 스키마로 제약된 Claude Sonnet 5 호출. 최종 판단 필드 이전에 내부 전용 필드(sender_trust, urgency_signal, risk_signal, repetition_signal, type_candidates)를 먼저 채우게 해 명시적 중간 추론을 강제.
5. 출력 + 평가	main.py, to_output_row() code/evaluation/	실패하더라도 모든 메시지가 output.csv에 정확히 1행씩 기록됨(안전한 fallback, 누락 행 없음). 별도 평가 워크플로우가 정답 30건 대비 예측을 채점.

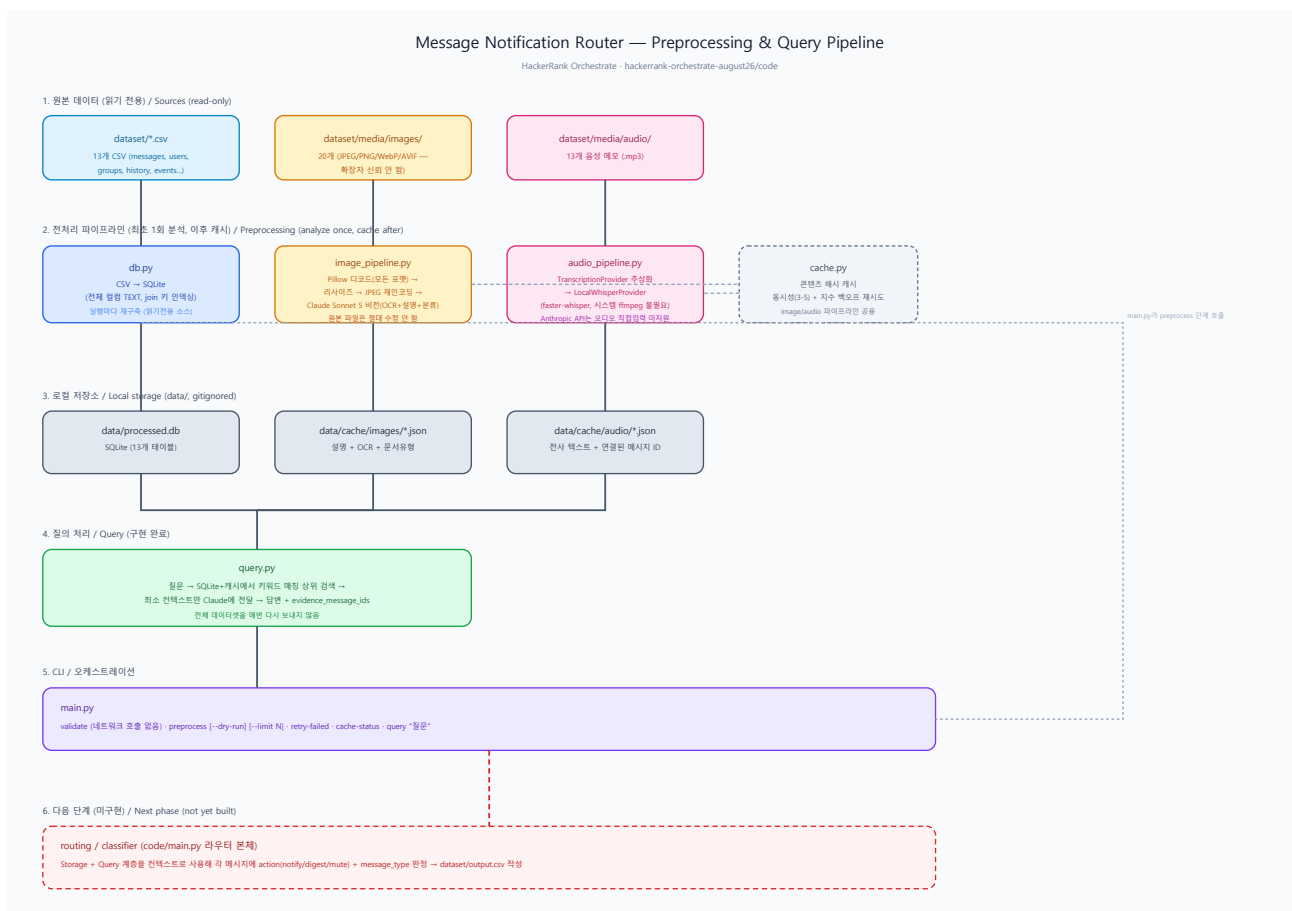


그림 1. 전처리-저장소-질의-라우팅 계층 (docs/pipeline.svg — 위에서 설명한 라우팅/평가 계층은 이 다이어그램이 보여주는 부분의 다음 단계에 위치).

4. 멀티모달 처리 — 주목할 만한 세부사항

- 포맷을 신뢰하지 않은 것이 실제로 도움이 됐다: 문제에서 알려준 3개뿐 아니라 이미지 20개 전체를 MIME 스니핑했더니 WebP/AVIF 사례 외에도 실제로는 PNG인 ".jpg" 파일이 여럿 발견됐다. 디코드 후 항상 JPEG로 재인코딩하는 방식이 포맷별 분기 없이 모든 경우를 일괄 처리했다.
- 오디오 입력 타입이 없다는 사실을 확인 후 결정: 이 빌드 시점 Anthropic Messages API에 오디오 콘텐츠 블록 타입이 없다는 걸 추측이 아니라 API 레퍼런스로 직접 확인한 뒤 접근법을 정했다. 로컬 faster-whisper(PyAV 내장, 시스템 ffmpeg 불필요)를 TranscriptionProvider 추상화 뒤에 두어, 나중에 호스팅형 ASR API로 클래스 하나만 바꿔 교체할 수 있게 설계.
- 콘텐츠 해시 캐싱(파일 바이트의 SHA-256)으로 바이트가 동일한 파일은 여러 번 실행해도 재분석하지 않음 — 파일명 변경은 재처리를 유발하지 않지만 실제 내용 변경은 자동으로 재처리됨.
- 동시성 제한 + 지수 백오프 재시도는 이미지/오디오 파이프라인이 공유하는 인프라(cache.py)로, 파이프라인마다 중복 구현하지 않음.

5. 분류 설계 — 중간 추론을 강제하기

초기 결과를 검토하며 두 가지 패턴이 드러났다: digest→notify가 가장 흔한 오답 방향이었고, 오답의 confidence가 정답보다 눈에 띄게 낮았다(합산 49건 기준 평균 0.70 vs 0.85). 해결책: JSON 스키마에 모델이 최종 판단 이전에 반드시 채워야 하는 내부 전용 필드를 추가 — output.csv에는 절대 기록되지 않지만, 판단이 먼저 나오고 사후에 합리화되는 대신 명시적 신호로부터 도출되도록 강제한다.

내부 필드	목적
sender_trust	verified_trusted_relationship → unverified_or_new_no_relationship (5단계)
urgency_signal	본문 자체가 긴급성을 명시/암시하는가(양방향)? "no rush" 같은 문구가 일정 언급만으로 무시되는 걸 방지
risk_signal	none → clear_scam_pattern — 기만/자격증명-결제정보 탈취 시도 신호
repetition_signal	이 메시지가 해당 발신자/주제에 대한 사용자 이력과 어떻게 연관되는가
type_candidates	최종 카테고리를 확정하기 전 고려하는 1~3개 후보 message_type과 상대적 가능성

여기에 관찰된 편향을 정확히 겨냥한 규칙 하나를 추가: "notify와 digest가 진짜로 애매하면 digest를 기본값으로 — 불확실하다는 것 자체는 인터럽트를 정당화하지 않는다." 30건 평가에서 검증된 종합 효과: action 정확도 86.67% → 93.33%, message_type 90% → 93.33%, 기존에 맞았던 26건에서 회귀 없음.

6. 주요 설계 결정

누적 의사결정 로그(전체는 CLAUDE.en.md/CLAUDE.ko.md §8) 중 선별한 항목 — 각각 기각한 대안과 이유를 포함:

결정	이유 / 기각한 대안
리사이즈 후 이미지는 원본 포맷과 무관하게 항상 JPEG로 재인코딩.	데이터셋의 ".jpg" 파일 여러 개가 실제로는 WebP/AVIF/PNG였음. 디코드 후 재인코딩하면 포맷별 분기 없이 모두 처리되며, 비전 API가 media_type으로 허용하지 않는 AVIF 문제도 함께 해결됨. 기각한 대안: 파일 확장자 신뢰.

결정	이유 / 기각한 대안
호스팅형 ASR API 대신 로컬 faster-whisper 사용.	Messages API에 오디오 콘텐츠 블록 타입이 없음을 확인했으므로 전사는 어차피 필수. 로컬 방식은 두 번째 API 키/비용을 피하고, PyAV 덕분에 이 머신의 "ffmpeg 없음" 문제도 우회. 기각한 대안: OpenAI Whisper API(여기선 정확도 이점 없이 비용/키만 추가).
시스템 기본 Python 3.14 대신 Python 3.12 venv 사용.	faster-whisper의 백엔드인 ctranslate2는 컴파일된 휠이라 최신 Python 출시보다 보통 몇 달 뒤처짐. 3.14는 시간제한 있는 해커톤에서 신뢰하기엔 너무 최신 버전. 결정 전 3.12가 깨끗이 설치됨을 확인. 기각한 대안: 3.14(휠 미지원 위험 실재).
라우팅 판단은 (미디어와 달리) 콘텐츠 해시가 아니라 message_id로 캐싱.	messages.csv는 실행 기간 동안 정적이고 message_id가 이미 각 행을 유일하게 식별함; 8개 이상 테이블을 조인한 전체 컨텍스트를 해시하는 건 이득 없는 복잡도. 기각한 대안: 콘텐츠 해시 키(여기선 과실계); 캐시 없음(테스트 실행마다 비용 재지불).
evidence_message_ids 검색은 임베딩이 아니라 수동 점수와 휴리스틱(발신자 > 그룹 > 비즈니스 > 최신순, 최대 12건).	데이터셋이 충분히 작아(사용자당 이력 최대 수백 건) 빠르고 설명 가능한 휴리스틱으로 충분하고 토큰 비용도 낮게 유지됨. 기각한 대안: 전체 이력 통째 전달(토큰 낭비, 신호 희석); 임베딩 검색(이 규모에서 불필요한 복잡도).
재시도 후에도 분류 실패 시, 메시지를 건너뛰거나 실행을 중단하는 대신 안전한 digest/unknown fallback 행을 출력.	제출 규약상 메시지당 정확히 1행이 있어야 하며, 행 누락보다는 명확히 표시된 저신뢰 대체값이 낫고, 캐시엔 실패로 남아 나중에 retry-failed가 실제로 재처리함. 기각한 대안: 행 생략; 전체 실행 중단.
message_type 세분화 규칙은 그럴듯한 정의를 추측하는 대신, 가장 오답률 높은 카테고리의 실제 오답 텍스트를 전부 읽어 도출.	이 데이터셋의 카테고리 경계는 라벨링 당시의 관습이지 카테고리 이름만으로 추론 가능한 게 아님(예: business 발신자의 예약 알림이 "event"로 분류되는 건 실제 사례를 보지 않고는 알 수 없음). 기각한 대안: 실제 예시 확인 없이 교과서적 정의만 작성.
30건 평가 수치를 최종으로 믿는 대신, 완전히 별개인 진짜 held-out 합성 테스트셋을 추가로 구축.	그때까지의 모든 프롬프트 수정이 sample_messages.csv 하나로만 검증됐고, 일반 규칙을 뺏으려 애써도 같은 30개 예시를 반복해서 읽는 것 자체가 과적합 위험을 만듦. 30건 세트에서는 드러나지 않았던 진짜 문제("not urgent" 프레이밍 과소반영)를 실제로 찾아냄. 기각한 대안: 30건 수치만 신뢰.
합성 테스트용 이미지/음성은 생성 API 대신 로컬(Pillow, Windows SAPI TTS)로 무료 생성.	채점 후 버려지는 평가용 입력이지 제출물이 아님 — 실제 데이터셋과 동일한 OCR/ASR/비전 코드 경로만 거치면 되고 미디어의 실사 품질은 중요하지 않음. 기각한 대안: 생성 API(채점 후 버려질 데이터에 불필요한 비용/키/지연).
key_phrase grounding 필드를 구현-테스트했다가, 실측된 퇴보(30건 평가 action 93.33%→86.67%) 확인 후 되돌림.	그렇듯해 보이는 grounding 기법이라고 다 도움이 되는 건 아님 — 범주형 내부 판단 필드는 효과가 있었지만 그 위에 자유 텍스트 추출을 얹은 건 아니었음. 저렴한 30건 검증 덕분에 제출 파일에 반영되기 전에 잡아냄. 기각한 대안: 재측정 없이 아이디어 자체의 그럴듯함만 믿고 유지.

7. 반복적 개발 과정 — 평가 루프

“만들고 → 돌리고 → 실패 사례 확인 → 고치고 → 다시 돌리기” 루프를 순서대로 정리. AI Judge 인터뷰가 가장 직접적으로 파고들 가능성이 높은 부분이다.

#	반복
1	첫 `route --limit 3` 테스트가 0/3 성공으로, 그것도 비정상적으로 빨리 끝남. 원인: ThreadPoolExecutor 워커들이 SQLite 커넥션 하나를 공유("다른 스레드에서 만든 객체는 그 스레드에서만 사용 가능"). 워커마다 자기 커넥션을 갖도록 수정.
2	재실행 → 3/3 성공. 규모를 키우기 전에 출력을 원본 텍스트와 직접 대조 확인 — 근거가 탄탄함을 확인(예: personal 대화로 위장한 OTP 피싱을, 같은 사용자가 과거 신고한 동일 메시지와 매칭해 정확히 잡아냄). 전체 110건 배치 실행.
3	sample_messages.csv가 messages.csv(msg_*)와 완전히 다른 id 체계(sample_msg_*)를 쓴다는 걸 발견 — output.csv를 id로 바로 비교하면 항상 0/30. sample_messages.csv 자신의 입력 필드로 라우터를 돌리는 스크립트를 새로 작성. 기준선: action 90%, message_type 63%, evidence F1 49%.

#	반복
4	실제 문제 하나 발견(옴트인한 반복 프로모션을 반복성만으로 과다 mute)하여 특정 사례가 아닌 일반 규칙으로 수정. action은 90% 유지(오답이 사라진 게 아니라 이동 — n=30 노이즈로 판단) — 보이는 세트에 과적합되지 않도록 의도적으로 튜닝 중단.
5	사용자가 message_type 정확도 개선을 명시적으로 요청. 가장 오답률 높은 카테고리(event, promotion, greeting)의 실제 오답 텍스트를 전부 읽고, 데이터의 실제 관습에서 일반화 가능한 세분화 규칙 4개를 도출. 결과: message_type 63%→90%.
6	프롬프트 튜닝에 한 번도 쓰인 적 없는 진짜 held-out 19건 합성 세트 구축. 첫 실행: action 74% / type 79% — 눈에 띄게 낮았고, 유의미한 원인 발견: "no rush"라고 명시했는데도 개인 메시지를 notify로 과다 판단. 일반 규칙 하나 더 추가해 재실행: 84%/84%, 30건 세트 회귀 없음.
7	사용자가 결과를 직접 검토하며 digest→notify 편향과 confidence-정확도 상관관계를 발견. 내부 추론 필드 + 불확실 시 digest 기본값 규칙 추가. 저렴하게 먼저 검증(이전 오답 7건: 3건 수정, message_type 회귀 0건) 후 30건 전체 확인: 93.33%/93.33%, 회귀 없음.
8	type_candidates에 key_phrase grounding 필드를 확장 시도. 결과: 실제 퇴보(action 93.33%→86.67%). 즉시 되돌려 최고 검증 버전을 복원하고, 그 버전으로 실제 메시지 110건 전체를 재분류.

8. 테스트 및 검증 전략

단일 신호를 맹신하지 않고, 세 가지 독립적인 신호를 의도적으로 함께 사용했다:

- 공식 정답 30건(dataset/sample_messages.csv) — 항상 사용 가능한 1차 로컬 대리 지표. 출력 스키마를 검증하고, 라벨 쓸림을 경고하며, action/message_type 혼동행렬을 출력하는 전용 하네스(code/evaluation/)로 채점.
- 진짜 held-out 합성 세트(19건: 텍스트 13 + Pillow로 그린 이미지 3 + Windows SAPI로 합성한 음성 3) — 프롬프트가 한 번도 본 적 없어 일반화 여부를 확인하기 위해 전용으로 구축. 30건 세트가 놓쳤던 긴급성-프레이밍 문제를 이 세트가 잡아냈다.
- 저렴한 타겟 재검증 — 프롬프트 변경 후 전체 재실행에 돈을 쓰기 전에, 이전에 틀렸던 것으로 이미 알려진 케이스만(30건이나 110건이 아니라 7건) 먼저 재분류해 아이디어가 통하는지 빠르고 저렴하게 확인.
- 스키마/정합성 검사 — 정확한 컬럼 순서, 허용된 action/message_type 값 집합, confidence 범위, evidence id가 실제 해당 사용자의 message_history에 존재하는지(다른 사용자 이력이나 허구의 id 아님), action 분포가 한쪽으로 쏠리지 않았는지.

합성 세트 19건 전체(이미지/오디오 삽입, expected-vs-actual, 성공/실패 표시)와 예시 삼아 만든 규칙 기반 “직접 시뮬레이션” 도구를 담은 HTML 리포트도 제작해 게시했다 — 이 세션에서 사용 가능한 아티팩트 플랫폼이 게시된 페이지에서 API 키 유출 위험 없이 Anthropic API를 직접 호출할 방법을 제공하지 않았기 때문에, 실제 모델 호출이 아니라 단순화된 클라이언트 사이드 근사치임을 명확히 라벨링했다.

9. 비용 및 리소스 관리

- 모델 선택(Claude Sonnet 5)은 지출 전에 명시적인 비용/품질/속도 추정을 거쳐 결정 — 데이터 규모가 작아(이미지 20장, 음성 13개, 메시지 110건) Opus급을 써도 절대 금액은 미미했지만, “유일하게 감당 가능한 선택지라서”가 아니라 낮은 비용에서 품질 여유를 확보하기 위해 Sonnet을 선택.
- 중요한 부분에서 로컬·무료 활용: Whisper ASR(호출당 API 비용 없음)과 합성 테스트 미디어 생성(Pillow + Windows SAPI TTS) — 둘 다 버려지는/개발 시점 비용을 0으로 유지.
- 파이프라인 전체에 걸친 콘텐츠 해시 캐싱 덕분에, 코드 수정 후 전처리를 다시 돌려도 변하지 않은 미디어에 대해서는 다시 비용을 지불하지 않음.
- 세션 중간에 실제 비용 추적 버그를 발견해 수정: cache-status 명령의 “누적 비용” 수치가 현재 캐시 파일만 합산했는데, 반복된 --force 재분류 실행이 매번 이 파일들을 덮어써서 여러 번의 전체 재실행에 걸친 총 비용을 조용히 과소보고하고 있었음. 각 실행이 보고한 비용을 하나하나 직접 합산해 발견했고, 이후 지출을 계속하기 전에 사용자에게 투명하게 알림.

10. 최종 결과

action 정확도	93.33% (30건 중 28건, 공식 정답 세트)
message_type 정확도	93.33% (30건 중 28건, 공식 정답 세트)
evidence F1 (평균)	~48–52%, 평가 실행마다 일관됨
합성 held-out 세트 (최종)	action 84% / message_type 84% (19건, 튜닝에 사용된 적 없음)
output.csv	110/110행 작성, fallback 행 0건, 스키마 정확히 일치
action 분포	mute 55 / notify 33 / digest 22 (편중 없음)

11. 알려진 한계 및 솔직한 미해결 사항

- evidence_message_ids F1(~48–52%)은 action/message_type 정확도보다 뒤처짐 — 현재의 휴리스틱 검색은 작동하지만 채점자가 가장 관련 있다고 볼 id와 항상 정확히 일치하진 않음. 30개 정답 세트에 과적합될 위험과 남은 예산 제약으로 더 튜닝하지 않음.
- spam vs scam, event vs urgent는 message_type 수정 이후에도 두 평가 세트 모두에서 계속 애매함 — 고칠 수 있는 버그라기보다(실제 내용상 양쪽 해석 다 타당함) 카테고리 경계 자체의 본질적 모호함으로 판단.
- Whisper 모델 크기는 base(가장 빠름, CPU 전용) — 더 큰 모델이 전사 품질을 개선할 수도 있지만 시도하지 않음; 표본 점검한 전사 결과가 이미 질적으로 충분했고 세션 막바지엔 예산도 빠듯했음.
- 내부 추론 필드는 경험적 개선이지 증명이 아니다 — 두 개의 독립적인 평가 세트에서 측정 가능한 도움이 됐고 이는 이 규모에서 의미 있는 근거이지만, n=30/n=19 평가 세트만으로는 히든 채점셋에 있을 수 있는 추가 엡지 케이스를 배제할 수 없음.

12. 제출물

- code.zip — code/ 디렉터리만(main.py, config.py, db.py, cache.py, image_pipeline.py, audio_pipeline.py, router.py, query.py, evaluation/), .venv/, data/, __pycache__/ 제외. code.zip 안의 내용만 검토자에게 보이므로 자체 완결된 code/README.md 포함.
- output.csv — dataset/messages.csv 110건 전체에 대한 최종 예측 결과.
- log.txt — AGENTS.md가 요구하는 채팅 트랜스크립트, 온보딩 동의와 이 보고서에 설명된 모든 결정을 포함해 세션 내내 매 턴마다 추가됨.

이 보고서 자체와 CLAUDE.en.md/CLAUDE.ko.md 전체 프로젝트 브리프(아키텍처 원칙, 전체 평가 루프 로그, 전체 의사결정 로그, 미해결 질문), 합성 테스트 HTML 리포트는 더 깊은 검토를 위해 저장소에 남아있지만, 제출 지침에 따라 code.zip에는 code/만 포함된다.